REACT

MODULE 1

Complete React Fundamentals

A detailed, classroom-ready guide with setup, directories, step-by-step coding, practical exercises, and complete solutions.

Stack: React + Vite + JavaScript + CSS
Format: Sessions → Concepts → Code → Practice → Solutions

For students • instructors • hands-on learning

How to Use This Module

This module is designed to be taught progressively. Students should type the examples rather than merely copy them. Watching code execute is educational; typing it, breaking it, and fixing it is where the brain finally admits it learned something.

Recommended flow

- **Instructor demonstration:** explain the idea briefly, then build the example live.
- **Student typing:** students recreate the example in their own project.
- **Debugging:** intentionally introduce one small mistake and diagnose it together.
- **Practice:** students complete the exercise without looking at the solution.
- **Solution:** compare approaches and discuss why the solution works.
- **Mini assignment:** extend the exercise with one additional feature.

Teaching rule

Do not rush through JSX and components just because they look easy. These are the grammar and sentence structure of React. Everything later builds on them.

Module Learning Outcomes

- Understand why React is used for interactive user interfaces.
- Create a React project using Vite and run the development server.
- Understand the purpose of the major project directories and files.
- Write JSX and understand how JSX differs from ordinary HTML.
- Use JavaScript expressions inside JSX.
- Create reusable functional components.
- Use props to pass data into components.
- Apply CSS to React components using practical patterns.
- Render lists using map() and use keys correctly.
- Build a small multi-component React application.
- Read common React errors and debug them systematically.

Module 1 Contents

Session 1 • React Introduction, Setup, Vite and Project Structure

Session 2 • JSX Fundamentals and JavaScript Inside JSX

Session 3 • Components and Component-Based Design

Session 4 • Props and Reusable Components

Session 5 • Styling, Lists, Keys and Rendering Data

Session 6 • Mini Project: Student Dashboard

Final Revision • Debugging, Common Mistakes, Assessment and Extension Tasks

Suggested duration: 6 teaching sessions + practice/assessment time.

SESSION 1

React Introduction, Setup, Vite & Project Structure

1.1 What is React?

React is a JavaScript library for building user interfaces. Instead of writing one enormous page and manually changing individual DOM nodes every time something changes, React lets us describe the UI as a collection of reusable components whose output depends on data.

A useful mental model

Traditional DOM approach: find an element → change it → find another element → update it → repeat until your code becomes a small archaeological site.

React approach: describe what the UI should look like for the current data → React updates the DOM when necessary.

> **Important**
>
> React does not replace JavaScript. React is JavaScript plus a component model and a rendering system. Strong JavaScript fundamentals make React much easier.

1.2 Why React?

- **Components:** split a large interface into small reusable pieces.
- **Declarative UI:** describe the desired UI rather than manually issuing DOM update instructions.
- **Reusable logic:** components can be reused with different data.
- **Large ecosystem:** routing, forms, data fetching, testing, UI libraries and many other tools are available.
- **Industry relevance:** React is widely used for web applications and front-end development.

1.3 React vs HTML/CSS/JavaScript

Technology	Main job
HTML	Structure and semantic content
CSS	Visual appearance and layout
JavaScript	Behavior, logic and data manipulation
React	Component-based UI construction and rendering

1.4 Prerequisites

- Basic HTML: headings, divs, buttons, forms and attributes.
- Basic CSS: selectors, classes, box model and flexbox.
- Basic JavaScript: variables, functions, arrays, objects, conditions and map().
- A terminal and a code editor such as VS Code.
- Node.js and npm installed.

1.5 Check Node.js and npm

Terminal

```
node -v
npm -v
```

Expected result

You should see version numbers. If the terminal says the command is not found, install Node.js before continuing. Do not randomly reinstall half the internet.

1.6 Create a React project with Vite

Terminal: create and run the project

```
npm create vite@latest react-module-1 -- --template react
cd react-module-1
npm install
npm run dev
```

The terminal will provide a local development URL. Open it in your browser.

What each command does

- **npm create vite@latest:** asks Vite to create a new project.
- **react-module-1:** the project folder name.
- **--template react:** choose the React JavaScript template.
- **cd:** enter the project directory.
- **npm install:** install dependencies listed in package.json.
- **npm run dev:** start the development server.

1.7 Understanding the Initial Directory

Project directory

```
react-module-1/
├── node_modules/
├── public/
│   └── vite.svg
├── src/
│   ├── assets/
│   │   └── react.svg
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .gitignore
├── index.html
├── package.json
├── package-lock.json
└── vite.config.js
```

Do not memorize the tree like a spelling test. Understand what each area is responsible for.

Directory/file guide

Path	Purpose
src/	Main application source code.
src/main.jsx	JavaScript entry point that renders the React application.
src/App.jsx	Root application component in the starter project.
src/index.css	Global styles.
src/App.css	Styles associated with the starter App component.
public/	Static assets that can be served directly.
package.json	Project metadata, dependencies and scripts.
vite.config.js	Vite configuration.
node_modules/	Installed dependencies. Do not manually edit it.

1.8 The React Entry Point

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
)
```

Breakdown

- **import:** brings functionality or components into the file.
- **createRoot:** creates the React root attached to an HTML element.
- **document.getElementById('root'):** finds the root element in index.html.
- **<App />:** renders the App component.
- **StrictMode:** enables additional development-time checks.

1.9 The HTML entry point

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>React Module 1</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Key connection

React does not magically replace the browser page. main.jsx connects the React application to the element.

1.10 First modification

src/App.jsx

```
function App() {
  return (
    <main>
      <h1>Hello React!</h1>
      <p>My first React application.</p>
    </main>
  )
}

export default App
```

Session 1 Practical Exercises

Exercise 1: Project Setup

Level: Beginner

- Create a Vite React project named student-react-app.
- Run it with npm run dev.
- Change the browser title in index.html.
- Replace the default App component with a simple welcome message.

Exercise 2: Profile Page

Level: Beginner

- Create a heading with your name.
- Add a paragraph describing yourself.
- Add three skills in an unordered list.
- Add a button labelled Contact Me.

Exercise 3: Understand the Tree

Level: Conceptual

- Identify which file starts the React application.
- Identify which file contains the root App component.
- Explain the role of package.json.
- Explain why node_modules should not be manually edited.

Session 1 Solutions

Solution 1: Project Setup

Terminal

```
npm create vite@latest student-react-app -- --template react
cd student-react-app
npm install
npm run dev
```

src/App.jsx

```
function App() {
  return (
    <main>
      <h1>Welcome to My React App</h1>
      <p>I am learning React with Vite.</p>
    </main>
  )
}

export default App
```

Solution 2: Profile Page

src/App.jsx

```
function App() {
  return (
    <main>
      <h1>Punit</h1>
      <p>I am learning React and building web applications.</p>

      <h2>Skills</h2>
      <ul>
        <li>JavaScript</li>
        <li>React</li>
        <li>CSS</li>
      </ul>

      <button>Contact Me</button>
    </main>
  )
}

export default App
```

Solution 3: Understand the Tree

- The application starts from **src/main.jsx**.
- The starter root component is **src/App.jsx**.
- **package.json** stores project metadata, dependencies and npm scripts.
- **node_modules** contains installed packages and should be managed by npm rather than manually edited.

SESSION 2

JSX Fundamentals & JavaScript Inside JSX

2.1 What is JSX?

JSX is a syntax extension commonly used with React. It lets us write markup-like structures directly inside JavaScript. JSX is not HTML, even though it looks remarkably similar. This is one of those things that seems obvious until a missing brace ruins your afternoon.

JSX with a JavaScript expression

```
function App() {
  const name = "Punit"

  return (
    <div>
      <h1>Hello {name}</h1>
      <p>Welcome to React.</p>
    </div>
  )
}
```

2.2 JSX rules you must know

- Return one parent element, or use a Fragment.
- Close every tag.
- Use className instead of class.
- Use camelCase for many DOM properties such as onClick.
- JavaScript expressions go inside curly braces: { }.
- Components must return valid JSX.

2.3 One parent element

JSX structure

```
// ■ Not valid as one return value
return (
  <h1>Hello</h1>
  <p>Welcome</p>
)

// ■ Wrapped in a parent
return (
  <div>
    <h1>Hello</h1>
    <p>Welcome</p>
  </div>
)

// ■ Fragment
return (
  <>
    <h1>Hello</h1>
    <p>Welcome</p>
  </>
)
```

2.4 JavaScript expressions in JSX

Expressions

```
function App() {
  const name = "Aman"
  const age = 20
  const isStudent = true

  return (
    <main>
      <h1>{name}</h1>
      <p>Age: {age}</p>
      <p>Student: {isStudent ? "Yes" : "No"}</p>
      <p>Next year: {age + 1}</p>
    </main>
  )
}
```

What can go inside { }?

- Variables
- Arithmetic expressions
- Function calls
- Ternary expressions
- Array methods such as map()
- Object property access

What should not be placed there?

- Full statements such as if blocks directly inside JSX.
- for loops directly inside JSX.
- Variable declarations directly inside JSX.

Move statements outside JSX

```
function App() {
  const age = 19
  let message

  if (age >= 18) {
    message = "Adult"
  } else {
    message = "Minor"
  }

  return <h1>{message}</h1>
}
```

2.5 JSX attributes

Dynamic attributes

```
function App() {
  const imageUrl = "https://example.com/photo.jpg"

  return (
    <img
      src={imageUrl}
      alt="Profile"
      width={200}
    />
  )
}
```

Remember

String values can be written directly as quoted attributes. JavaScript expressions use curly braces.

2.6 Conditional rendering

Ternary conditional

```
function App() {
  const isLoggedIn = true

  return (
    <main>
      {isLoggedIn ? (
        <h1>Welcome back!</h1>
      ) : (
        <h1>Please log in.</h1>
      )}
    </main>
  )
}
```

Logical AND

&& conditional rendering

```
function App() {
  const hasNotification = true

  return (
    <div>
      <h1>Dashboard</h1>
      {hasNotification && <p>You have new notifications.</p>}
    </div>
  )
}
```

2.7 Rendering arrays

Array → JSX

```
function App() {
  const skills = ["HTML", "CSS", "JavaScript", "React"]

  return (
    <ul>
      {skills.map((skill) => (
        <li key={skill}>{skill}</li>
      ))}
    </ul>
  )
}
```

Session 2 Practical Exercises

Exercise 4: Personal Information Card

Level: Beginner

- Create name, age, city and course variables.
- Display them using JSX expressions.
- Calculate and display the student's age next year.
- Display 'Student' or 'Not a student' using a ternary.

Exercise 5: Skill List

Level: Beginner

- Create an array containing five skills.
- Render the skills as elements using map().
- Add a key to each list item.

Exercise 6: Status Message

Level: Beginner+

- Create a boolean named isOnline.
- Show 'Online' when true and 'Offline' when false.
- Also show a small message only when the user is online.

Session 2 Solutions

Solution 4: Personal Information Card

App.jsx

```
function App() {
  const name = "Punit"
  const age = 20
  const city = "Kaithal"
  const course = "React Development"
  const isStudent = true

  return (
    <main>
      <h1>{name}</h1>
      <p>Age: {age}</p>
      <p>City: {city}</p>
      <p>Course: {course}</p>
      <p>Next year: {age + 1}</p>
      <p>{isStudent ? "Student" : "Not a student"}</p>
    </main>
  )
}

export default App
```

Solution 5: Skill List

App.jsx

```
function App() {
  const skills = [
    "HTML",
    "CSS",
    "JavaScript",
    "React",
    "Git"
  ]

  return (
    <ul>
      {skills.map((skill) => (
        <li key={skill}>{skill}</li>
      ))}
    </ul>
  )
}

export default App
```

Solution 6: Status Message

App.jsx

```
function App() {
  const isOnline = true

  return (
    <main>
      <h1>{isOnline ? "Online" : "Offline"}</h1>

      {isOnline && (
        <p>The user can receive notifications.</p>
      )}
    </main>
  )
}

export default App
```

SESSION 3

Components & Component-Based Design

3.1 What is a component?

A component is a reusable piece of UI. In modern React, components are commonly JavaScript functions that return JSX. A good component has a clear responsibility and can often be understood without reading the entire application.

First component

```
function Welcome() {
  return <h1>Welcome to React!</h1>
}

export default Welcome
```

3.2 Why components matter

- Break large interfaces into smaller pieces.
- Reuse UI patterns.
- Make code easier to test and maintain.
- Keep responsibilities separated.
- Make future changes local instead of turning App.jsx into a 900-line monument to poor planning.

3.3 Naming rules

- Component names should begin with a capital letter.
- Use PascalCase for component names: UserCard, Navbar, ProductList.
- Keep component names descriptive.
- One component should have a focused responsibility.

3.4 Create a components directory

Improved structure

```
src/
■■■ components/
■   ■■■ Header.jsx
■   ■■■ Welcome.jsx
■   ■■■ Footer.jsx
■■■ App.jsx
■■■ main.jsx
■■■ index.css
```

3.5 Creating and importing components

Header.jsx

```
// src/components/Header.jsx

function Header() {
  return (
    <header>
      <h1>My Website</h1>
      <nav>
        <a href="#">Home</a>
        <a href="#">About</a>
      </nav>
    </header>
  )
}

export default Header
```

App.jsx

```
// src/App.jsx

import Header from "./components/Header"

function App() {
  return (
    <>
      <Header />
      <main>
        <h2>Home Page</h2>
      </main>
    </>
  )
}

export default App
```

3.6 Multiple components

Two separate components

```
// src/components/Welcome.jsx
function Welcome() {
  return <p>Learn React one component at a time.</p>
}

export default Welcome

// src/components/Footer.jsx
function Footer() {
  return <footer>© 2026 My App</footer>
}

export default Footer
```

Composing components

```
// src/App.jsx
import Header from "./components/Header"
import Welcome from "./components/Welcome"
import Footer from "./components/Footer"

function App() {
  return (
    <>
      <Header />
      <main>
        <Welcome />
      </main>
      <Footer />
    </>
  )
}

export default App
```

3.7 Component hierarchy

Example component tree

```
App
■■■ Header
■   ■■■ Navigation
■■■ Main
■   ■■■ Welcome
■   ■■■ FeatureList
■■■ Footer
```

The important idea is parent-child relationships. App can render Header, Main and Footer. Main can render other components. Later, data can move between these components using props.

3.8 Avoiding the giant App.jsx

For small practice projects, App.jsx can contain several things. As an application grows, separate components by responsibility. App.jsx can remain the composition layer while individual components handle their own UI.

A scalable starting structure

```
src/
■■■ components/
■   ■■■ Navbar.jsx
■   ■■■ Hero.jsx
■   ■■■ FeatureCard.jsx
■   ■■■ FeatureList.jsx
■   ■■■ Footer.jsx
■■■ pages/
■   ■■■ Home.jsx
■■■ App.jsx
■■■ main.jsx
```

Practical rule

Do not create 40 folders for a three-component project. Structure should reduce complexity, not become a hobby.

Session 3 Practical Exercises

Exercise 7: Three Component Website

Level: Intermediate

- Create Header.jsx, MainContent.jsx and Footer.jsx.
- Render all three from App.jsx.
- Header should contain a title and navigation.
- MainContent should contain a heading and paragraph.
- Footer should contain a copyright message.

Exercise 8: Product Card Component

Level: Intermediate

- Create ProductCard.jsx.
- Display a product name, price and description.
- Render the component three times from App.jsx.
- Use different hard-coded values initially.

Session 3 Solutions

Solution 7: Three Component Website

Header.jsx

```
// src/components/Header.jsx
function Header() {
  return (
    <header>
      <h1>My Website</h1>
      <nav>
        <a href="#">Home</a>
        <a href="#">Courses</a>
        <a href="#">Contact</a>
      </nav>
    </header>
  )
}

export default Header
```

MainContent.jsx

```
// src/components/MainContent.jsx
function MainContent() {
  return (
    <main>
      <h2>Welcome</h2>
      <p>This website is built with React.</p>
    </main>
  )
}

export default MainContent
```

Footer.jsx

```
// src/components/Footer.jsx
function Footer() {
  return <footer>© 2026 My Website</footer>
}

export default Footer
```

App.jsx

```
// src/App.jsx
import Header from "./components/Header"
import MainContent from "./components/MainContent"
import Footer from "./components/Footer"

function App() {
  return (
    <>
      <Header />
      <MainContent />
      <Footer />
    </>
  )
}

export default App
```

Solution 8: Product Card

ProductCard.jsx

```
// src/components/ProductCard.jsx
function ProductCard() {
  return (
    <article>
      <h2>JavaScript Book</h2>
      <p>Learn JavaScript from basics to advanced topics.</p>
      <strong>■499</strong>
    </article>
  )
}

export default ProductCard
```

App.jsx

```
// App.jsx
import ProductCard from "./components/ProductCard"

function App() {
  return (
    <main>
      <ProductCard />
      <ProductCard />
      <ProductCard />
    </main>
  )
}

export default App
```

SESSION 4

Props & Reusable Components

4.1 The problem with hard-coded components

Our ProductCard works, but every instance displays the same product. Reusability becomes meaningful when a component can receive different data. React uses **props** for this.

4.2 Passing props

Component receiving props

```
function ProductCard({ name, price, description }) {
  return (
    <article>
      <h2>{name}</h2>
      <p>{description}</p>
      <strong>■{price}</strong>
    </article>
  )
}

export default ProductCard
```

Passing props

```
import ProductCard from "./components/ProductCard"

function App() {
  return (
    <main>
      <ProductCard
        name="JavaScript Book"
        price={499}
        description="Learn JavaScript fundamentals."
      />

      <ProductCard
        name="React Book"
        price={699}
        description="Build interfaces with React."
      />
    </main>
  )
}

export default App
```

4.3 Props are read-only

A child component should not directly modify its received props. Props represent data supplied by the parent. This makes the data flow predictable.

Data flow

A common beginner mental model is: Parent → Props → Child. Later, events can let a child communicate an action back to a parent, but the data model remains explicit.

4.4 Props can contain many data types

Props of different types

```
function UserCard({ name, age, isStudent, skills }) {
  return (
    <article>
      <h2>{name}</h2>
      <p>Age: {age}</p>
      <p>{isStudent ? "Student" : "Not a student"}</p>

      <ul>
        {skills.map((skill) => (
          <li key={skill}>{skill}</li>
        ))}
      </ul>
    </article>
  )
}

export default UserCard
```

Passing string, number, boolean and array props

```
<UserCard
  name="Riya"
  age={21}
  isStudent={true}
  skills={["HTML", "CSS", "React"]}
/>
```

4.5 Passing an object as a prop

Object prop

```
const user = {
  name: "Riya",
  age: 21,
  city: "Kaithal"
}

<UserCard user={user} />
```

Reading object prop

```
function UserCard({ user }) {
  return (
    <article>
      <h2>{user.name}</h2>
      <p>Age: {user.age}</p>
      <p>City: {user.city}</p>
    </article>
  )
}
```

4.6 children prop

children

```
function Card({ children }) {
  return (
    <section className="card">
      {children}
    </section>
  )
}

export default Card
```

Using children

```
<Card>
  <h2>React</h2>
  <p>Component-based UI development.</p>
</Card>
```

4.7 A reusable Button component

Reusable Button

```
function Button({ text, type = "button" }) {
  return (
    <button type={type}>
      {text}
    </button>
  )
}

export default Button
```

Different configurations

```
<Button text="Save" />
<Button text="Delete" />
<Button text="Submit" type="submit" />
```

4.8 Props destructuring

Two equivalent styles

```
// Without destructuring
function UserCard(props) {
  return <h2>{props.name}</h2>
}

// With destructuring
function UserCard({ name }) {
  return <h2>{name}</h2>
}
```

4.9 Default values

Default prop values

```
function Badge({ label = "New", color = "blue" }) {
  return (
    <span className={color}>
      {label}
    </span>
  )
}
```

Session 4 Practical Exercises

Exercise 9: Reusable Student Card

Level: Intermediate

- Create StudentCard.jsx.
- Accept name, course, age and skills as props.
- Render at least three students.
- Render the skills using map().

Exercise 10: Reusable Button

Level: Intermediate

- Create a Button component.
- Accept text and variant as props.
- Render at least three buttons with different labels.
- Use a class name based on the variant.

Exercise 11: Card with Children

Level: Intermediate+

- Create a Card component that accepts children.
- Use it for a course card and a notice card.
- Do not hard-code the inside content in Card.jsx.

Session 4 Solutions

Solution 9: Reusable Student Card

StudentCard.jsx

```
// StudentCard.jsx
function StudentCard({ name, course, age, skills }) {
  return (
    <article className="student-card">
      <h2>{name}</h2>
      <p>Course: {course}</p>
      <p>Age: {age}</p>

      <h3>Skills</h3>
      <ul>
        {skills.map((skill) => (
          <li key={skill}>{skill}</li>
        ))}
      </ul>
    </article>
  )
}

export default StudentCard
```

App.jsx

```
// App.jsx
import StudentCard from "./components/StudentCard"

function App() {
  return (
    <main>
      <StudentCard
        name="Riya"
        course="React"
        age={20}
        skills={["HTML", "CSS", "JS"]}
      />

      <StudentCard
        name="Aman"
        course="MERN"
        age={22}
        skills={["React", "Node", "MongoDB"]}
      />

      <StudentCard
        name="Neha"
        course="Frontend"
        age={19}
        skills={["HTML", "CSS", "JavaScript"]}
      />
    </main>
  )
```

App.jsx (continued)

```
}

export default App
```

Solution 10: Reusable Button

Button.jsx

```
function Button({ text, variant = "primary" }) {
  return (
    <button className={`btn ${variant}`}>
      {text}
    </button>
  )
}

export default Button
```

Using Button

```
<Button text="Save" variant="primary" />
<Button text="Delete" variant="danger" />
<Button text="Cancel" variant="secondary" />
```

Solution 11: Card with Children

Card.jsx

```
function Card({ children }) {
  return (
    <section className="card">
      {children}
    </section>
  )
}

export default Card
```

Using Card

```
<Card>
  <h2>React Course</h2>
  <p>Learn components, props and JSX.</p>
</Card>

<Card>
  <h2>Notice</h2>
  <p>Assignment submission is Friday.</p>
</Card>
```

SESSION 5

Styling, Lists, Keys & Rendering Data

5.1 Styling React components

React does not force a single styling method. For Module 1 we will use ordinary CSS because students need to understand the underlying mechanism before adding another abstraction layer.

5.2 CSS file import

Importing CSS

```
// App.jsx
import "./App.css"

function App() {
  return (
    <main className="app">
      <h1 className="title">React Course</h1>
    </main>
  )
}

export default App
```

App.css

```
.app {
  max-width: 900px;
  margin: 0 auto;
  padding: 2rem;
}

.title {
  color: #2563eb;
  font-size: 2rem;
}
```

5.3 className

className instead of class

```
function Card() {
  return (
    <article className="card">
      <h2 className="card-title">React</h2>
    </article>
  )
}
```

Why className?

class is a JavaScript keyword, so JSX uses className for CSS classes.

5.4 Dynamic class names

Conditional className

```
function Status({ online }) {
  return (
    <p className={online ? "status online" : "status offline"}>
      {online ? "Online" : "Offline"}
    </p>
  )
}
```

CSS

```
.status {
  font-weight: 700;
}

.online {
  color: green;
}

.offline {
  color: red;
}
```

5.5 Inline styles

Style object

```
function App() {
  const headingStyle = {
    color: "purple",
    fontSize: "32px"
  }

  return <h1 style={headingStyle}>React</h1>
}
```

Use carefully

Inline styles are useful for small dynamic values, but a CSS file is often clearer for larger styling systems.

5.6 Rendering arrays with map()

List rendering

```
const courses = [
  "HTML",
  "CSS",
  "JavaScript",
  "React"
]

function CourseList() {
  return (
    <ul>
      {courses.map((course) => (
        <li key={course}>{course}</li>
      ))}
    </ul>
  )
}
```

5.7 Why keys matter

When React renders a list, each item needs a stable key so React can identify which item corresponds to which rendered element when the list changes.

Stable key

```
// Good when id is stable and unique
{students.map((student) => (
  <StudentCard
    key={student.id}
    name={student.name}
  />
))}
```

Index key: usually not ideal for dynamic lists

```
// Avoid when list order can change
{students.map((student, index) => (
  <StudentCard
    key={index}
    name={student.name}
  />
))}
```

5.8 Rendering objects

Object list

```
const students = [
  {
    id: 1,
    name: "Riya",
    course: "React"
  },
  {
    id: 2,
    name: "Aman",
    course: "MERN"
  }
]

function StudentList() {
  return (
    <div>
      {students.map((student) => (
        <article key={student.id}>
          <h2>{student.name}</h2>
          <p>{student.course}</p>
        </article>
      ))}
    </div>
  )
}
```

5.9 Conditional list content

Empty-state handling

```
function CourseList({ courses }) {
  return (
    <section>
      <h2>Courses</h2>

      {courses.length === 0 ? (
        <p>No courses available.</p>
      ) : (
        <ul>
          {courses.map((course) => (
            <li key={course.id}>{course.name}</li>
          ))}
        </ul>
      )}
    </section>
  )
}
```

5.10 Filtering before rendering

filter + map

```
const products = [
  { id: 1, name: "Book", inStock: true },
  { id: 2, name: "Pen", inStock: false },
  { id: 3, name: "Notebook", inStock: true }
]

function ProductList() {
  const available = products.filter(
    (product) => product.inStock
  )

  return (
    <ul>
      {available.map((product) => (
        <li key={product.id}>
          {product.name}
        </li>
      ))}
    </ul>
  )
}
```

Session 5 Practical Exercises

Exercise 12: Styled Product Cards

Level: Intermediate

- Create a ProductCard component.
- Create CSS for card, title, description and price.
- Render three cards.
- Use a different class for an 'on sale' product.

Exercise 13: Course List

Level: Intermediate

- Create an array of course objects with id, name and duration.
- Render the courses using map().
- Use id as the key.
- Display a message if there are no courses.

Exercise 14: Available Products

Level: Intermediate+

- Create five products.
- Each product should have inStock true or false.
- Use filter() to show only available products.
- Render the result using map().

Session 5 Solutions

Solution 12: Styled Product Cards

ProductCard.jsx

```
// ProductCard.jsx
function ProductCard({
  name,
  description,
  price,
  sale
}) {
  return (
    <article className={`card ${sale ? "sale" : ""}`}>
      <h2>{name}</h2>
      <p>{description}</p>
      <strong>■{price}</strong>
      {sale && <span>SALE</span>}
    </article>
  )
}

export default ProductCard
```

App.css

```
.card {
  padding: 20px;
  border: 1px solid #ddd;
  border-radius: 12px;
  margin-bottom: 12px;
}

.card h2 {
  margin-top: 0;
}

.sale {
  border-color: #ea580c;
  background: #fff7ed;
}

.sale span {
  display: inline-block;
  margin-left: 12px;
  font-weight: bold;
  color: #ea580c;
}
```

Solution 13: Course List

CourseList.jsx

```
function CourseList({ courses }) {
  if (courses.length === 0) {
    return <p>No courses available.</p>
  }

  return (
    <ul>
      {courses.map((course) => (
        <li key={course.id}>
          {course.name} - {course.duration}
        </li>
      ))}
    </ul>
  )
}

export default CourseList
```

Solution 14: Available Products

ProductList.jsx

```
const products = [
  { id: 1, name: "Book", inStock: true },
  { id: 2, name: "Pen", inStock: false },
  { id: 3, name: "Notebook", inStock: true },
  { id: 4, name: "Bag", inStock: false },
  { id: 5, name: "Bottle", inStock: true }
]

function ProductList() {
  const availableProducts = products.filter(
    (product) => product.inStock
  )

  return (
    <ul>
      {availableProducts.map((product) => (
        <li key={product.id}>{product.name}</li>
      ))}
    </ul>
  )
}

export default ProductList
```

SESSION 6

Mini Project: Student Dashboard

6.1 Project goal

We will combine the ideas from Module 1 into a small Student Dashboard. It will use components, props, arrays, map(), keys, conditional rendering and CSS. We will deliberately keep state and API calls out of this module so the architecture remains understandable.

6.2 Final directory

Project structure

```
student-dashboard/
■■■ public/
■■■ src/
■   ■■■ components/
■   ■   ■■■ Header.jsx
■   ■   ■■■ StudentCard.jsx
■   ■   ■■■ StudentList.jsx
■   ■   ■■■ Stats.jsx
■   ■■■ App.jsx
■   ■■■ App.css
■   ■■■ index.css
■   ■■■ main.jsx
■■■ package.json
■■■ vite.config.js
```

6.3 Step 1: Create the project

Terminal

```
npm create vite@latest student-dashboard -- --template react
cd student-dashboard
npm install
npm run dev
```

6.4 Step 2: Create the components folder

Terminal (optional)

```
mkdir src/components
```

6.5 Step 3: Header component

Header.jsx

```
// src/components/Header.jsx
function Header() {
  return (
    <header className="header">
      <h1>Student Dashboard</h1>
      <p>React Module 1 Project</p>
    </header>
  )
}

export default Header
```

6.6 Step 4: StudentCard component

StudentCard.jsx

```
// src/components/StudentCard.jsx
function StudentCard({ student }) {
  return (
    <article className="student-card">
      <h2>{student.name}</h2>
      <p>Course: {student.course}</p>
      <p>Age: {student.age}</p>

      <h3>Skills</h3>
      <ul>
        {student.skills.map((skill) => (
          <li key={skill}>{skill}</li>
        ))}
      </ul>

      <p className={student.active ? "active" : "inactive"}>
        {student.active ? "Active" : "Inactive"}
      </p>
    </article>
  )
}

export default StudentCard
```

6.7 Step 5: StudentList component

StudentList.jsx

```
// src/components/StudentList.jsx
import StudentCard from "./StudentCard"

function StudentList({ students }) {
  if (students.length === 0) {
    return <p>No students found.</p>
  }

  return (
    <section>
      <h2>Students</h2>

      <div className="student-grid">
        {students.map((student) => (
          <StudentCard
            key={student.id}
            student={student}
          />
        ))}
      </div>
    </section>
  )
}

export default StudentList
```

6.8 Step 6: Stats component

Stats.jsx

```
// src/components/Stats.jsx
function Stats({ students }) {
  const activeStudents = students.filter(
    (student) => student.active
  )

  return (
    <section className="stats">
      <div>
        <strong>{students.length}</strong>
        <span>Total Students</span>
      </div>

      <div>
        <strong>{activeStudents.length}</strong>
        <span>Active Students</span>
      </div>
    </section>
  )
}

export default Stats
```

6.9 Step 7: App component

App.jsx

```
// src/App.jsx
import Header from "./components/Header"
import Stats from "./components/Stats"
import StudentList from "./components/StudentList"
import "./App.css"

const students = [
  {
    id: 1,
    name: "Riya",
    course: "React",
    age: 20,
    active: true,
    skills: ["HTML", "CSS", "JS"]
  },
  {
    id: 2,
    name: "Aman",
    course: "MERN",
    age: 22,
    active: true,
    skills: ["React", "Node", "MongoDB"]
  },
  {
    id: 3,
    name: "Neha",
    course: "Frontend",
    age: 19,
```

App.jsx (continued)

```
    active: false,
    skills: ["HTML", "CSS"]
  }
]

function App() {
  return (
    <div className="app">
      <Header />
      <Stats students={students} />
      <StudentList students={students} />
    </div>
  )
}

export default App
```

6.10 Step 8: CSS

App.css

```
.app {
  max-width: 1000px;
  margin: 0 auto;
  padding: 24px;
  font-family: Arial, sans-serif;
}

.header {
  padding: 24px;
  border-radius: 16px;
  background: #172554;
  color: white;
  margin-bottom: 20px;
}

.stats {
  display: flex;
  gap: 16px;
  margin-bottom: 20px;
}

.stats div {
  flex: 1;
  padding: 20px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
}
```

App.css (continued)

```
.stats strong,
.stats span {
  display: block;
}

.stats strong {
  font-size: 28px;
}

.student-grid {
  display: grid;
  grid-template-columns: repeat(
    auto-fit,
    minmax(220px, 1fr)
  );
  gap: 16px;
}

.student-card {
  padding: 18px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  background: white;
}

.active {
  color: #059669;
  font-weight: bold;
```

App.css (continued)

```
}

.inactive {
  color: #dc2626;
  font-weight: bold;
}
```

6.11 Step 9: Global CSS

index.css

```
/* src/index.css */

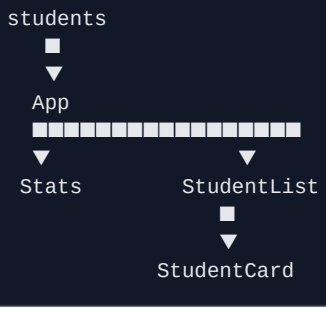
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: #f8fafc;
  color: #172033;
}

button,
input {
  font: inherit;
}
```

6.12 How data flows

Data flow



App owns the hard-coded data for this project. It passes the same students array to Stats and StudentList. StudentList passes one student object to each StudentCard. This is the basic parent-to-child data flow students need before learning state management.

Checkpoint

At this point students should be able to explain every arrow in the component tree. If they cannot, do not jump to hooks yet. React will still be there tomorrow.

6.13 Mini Project Extension Tasks

- Add a city property to each student and display it.
- Add a course duration property.
- Create a CourseCard component.
- Add a footer component.
- Display a message when there are no students.
- Add a 'Top Skill' field and display it.

• Create a reusable Badge component for Active/Inactive.

Final Revision: Debugging & Common Mistakes

React beginners often think an error means React is difficult. Usually it means the browser has been given something syntactically impossible, a file path is wrong, or a component was imported incorrectly. Computers remain aggressively literal.

Component starts with lowercase

Common mistake

```
function header() {
  return <h1>Header</h1>
}
```

Fix

Use PascalCase for components: function Header() {}

Forgot export

Common mistake

```
function Header() {
  return <h1>Header</h1>
}
```

Fix

Add export default Header if importing it as a default export.

Wrong import path

Common mistake

```
import Header from "./Header"
```

Fix

Check the actual file location. If Header.jsx is inside components, use ./components/Header.

Using class instead of className

Common mistake

```
<div class="card">Hello</div>
```

Fix

Use Hello.

Missing key in a list

Common mistake

```
items.map((item) => <li>{item}</li>)
```

Fix

Add a stable key: {item}.

Trying to use statements inside JSX

Common mistake

```
return (
  <div>
    {if (loggedIn) ...}
  </div>
)
```

Fix

Use a ternary, &&, or calculate the value before return.

Module 1 Revision Questions

1. What is React?
2. Why are components useful?
3. What command creates a React project with Vite?
4. What does npm install do?
5. What is the purpose of src/main.jsx?
6. What is JSX?
7. Why do we use className instead of class?
8. What are JavaScript expressions in JSX?
9. What is a component?
10. What is the difference between a component and an HTML element?
11. What are props?
12. Are props directly mutable by the child component?
13. What is the children prop?
14. Why does a mapped list need a key?
15. Why is a stable id usually better than an array index as a key?
16. What does map() return?
17. What does filter() return?
18. How does data normally flow from parent to child?
19. Why should large applications be split into components?
20. What is the purpose of package.json?

Module 1 Final Practical Test

Build a **Course Dashboard** from scratch. Do not copy the Student Dashboard project. Use the same concepts in a new context.

Requirements

- Create a Vite React project named course-dashboard.
- Create components: Header, Stats, CourseList, CourseCard and Footer.
- Create an array of at least four course objects.
- Each course must have id, title, instructor, duration, level and active.
- Pass the courses array from App to Stats and CourseList using props.
- Render cards with map().
- Use the course id as key.
- Show Active or Inactive status using conditional rendering.
- Use CSS to create a responsive grid.
- Show 'No courses available' when the array is empty.
- Keep the project organized using a components directory.

Expected directory

Target structure

```
course-dashboard/
■■■ src/
    ■■■ components/
    ■   ■■■ Header.jsx
    ■   ■■■ Stats.jsx
    ■   ■■■ CourseList.jsx
    ■   ■■■ CourseCard.jsx
    ■   ■■■ Footer.jsx
    ■■■ App.jsx
    ■■■ App.css
    ■■■ index.css
    ■■■ main.jsx
```

Self-check checklist

- The app runs with npm run dev.
- No console errors.
- No missing key warnings.
- Components are separated logically.
- Props are used rather than duplicating hard-coded cards.
- The empty list state works.
- The layout is readable on narrow screens.

Final Project Solution

Use this only after attempting the project yourself. Otherwise you are merely outsourcing the thinking and calling it learning.

Header.jsx

```
// src/components/Header.jsx
function Header() {
  return (
    <header className="header">
      <h1>Course Dashboard</h1>
      <p>Learn. Build. Repeat.</p>
    </header>
  )
}

export default Header
```

CourseCard.jsx

```
// src/components/CourseCard.jsx
function CourseCard({ course }) {
  return (
    <article className="course-card">
      <h2>{course.title}</h2>
      <p>Instructor: {course.instructor}</p>
      <p>Duration: {course.duration}</p>
      <p>Level: {course.level}</p>

      <span className={course.active ? "active" : "inactive"}>
        {course.active ? "Active" : "Inactive"}
      </span>
    </article>
  )
}

export default CourseCard
```

CourseList.jsx

```
// src/components/CourseList.jsx
import CourseCard from "./CourseCard"

function CourseList({ courses }) {
  if (courses.length === 0) {
    return <p>No courses available.</p>
  }

  return (
    <section>
      <h2>Courses</h2>

      <div className="course-grid">
        {courses.map((course) => (
          <CourseCard
            key={course.id}
            course={course}
          />
        ))}
      </div>
    </section>
  )
}

export default CourseList
```

Final Project Solution: Stats & App

Stats.jsx

```
// src/components/Stats.jsx
function Stats({ courses }) {
  const active = courses.filter(
    (course) => course.active
  ).length

  return (
    <section className="stats">
      <div>
        <strong>{courses.length}</strong>
        <span>Total Courses</span>
      </div>

      <div>
        <strong>{active}</strong>
        <span>Active Courses</span>
      </div>
    </section>
  )
}

export default Stats
```

Footer.jsx

```
// src/components/Footer.jsx
function Footer() {
  return (
    <footer className="footer">
      <p>React Module 1 • Course Dashboard</p>
    </footer>
  )
}

export default Footer
```

App.jsx

```
// src/App.jsx
import Header from "./components/Header"
import Stats from "./components/Stats"
import CourseList from "./components/CourseList"
import Footer from "./components/Footer"
import "./App.css"

const courses = [
  {
    id: 1,
    title: "HTML & CSS",
    instructor: "Asha",
    duration: "4 weeks",
    level: "Beginner",
    active: true
  },
  {
    id: 2,
    title: "JavaScript",
    instructor: "Rahul",
    duration: "6 weeks",
    level: "Intermediate",
    active: true
  },
  {
    id: 3,
    title: "React",
    instructor: "Neha",
```

App.jsx (continued)

```
    duration: "6 weeks",
    level: "Intermediate",
    active: true
  },
  {
    id: 4,
    title: "Node.js",
    instructor: "Vikram",
    duration: "5 weeks",
    level: "Intermediate",
    active: false
  }
]

function App() {
  return (
    <div className="app">
      <Header />
      <Stats courses={courses} />
      <CourseList courses={courses} />
      <Footer />
    </div>
  )
}

export default App
```

Final Project Solution: Styling

App.css

```
.app {
  max-width: 1000px;
  margin: 0 auto;
  padding: 24px;
  font-family: Arial, sans-serif;
}

.header {
  padding: 28px;
  margin-bottom: 20px;
  border-radius: 18px;
  background: #172554;
  color: white;
}

.stats {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
  margin-bottom: 24px;
}

.stats div {
  padding: 20px;
  border: 1px solid #cbd5e1;
  border-radius: 14px;
  background: white;
}
```

App.css (continued)

```
.stats strong,
.stats span {
  display: block;
}

.stats strong {
  font-size: 28px;
  margin-bottom: 4px;
}

.course-grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(230px, 1fr));
  gap: 16px;
}

.course-card {
  padding: 20px;
  border: 1px solid #cbd5e1;
  border-radius: 14px;
  background: white;
}

.course-card h2 {
  margin-top: 0;
}
```

App.css (continued)

```
.active {
  color: #059669;
  font-weight: bold;
}

.inactive {
  color: #dc2626;
  font-weight: bold;
}

.footer {
  margin-top: 30px;
  padding: 20px;
  text-align: center;
  color: #64748b;
}

@media (max-width: 600px) {
  .stats {
    grid-template-columns: 1fr;
  }
}
```

Module 1 mastery checklist

- I can create and run a React project with Vite.
- I understand the src directory and entry point.
- I can write valid JSX.
- I can use JavaScript expressions in JSX.
- I can split an interface into components.
- I can import and export components.
- I can pass data using props.
- I can render arrays with map().
- I understand why keys are required.
- I can conditionally render UI.
- I can style components with CSS.
- I can build a small multi-component application.

Instructor Notes: Suggested Teaching Sequence

Session	Topic	Time	Teaching focus
Session 1	React + Vite setup	40–60 min	Live-create a project. Spend time explaining main.jsx, App.jsx and package.json.
Session 2	JSX	45–60 min	Focus on expressions, attributes, conditions and arrays. Make students type every example.
Session 3	Components	45–60 min	Build a small page from Header, Main and Footer. Discuss component responsibility.
Session 4	Props	60 min	Turn hard-coded cards into reusable components. Props should feel necessary, not theoretical.
Session 5	CSS + lists	60 min	Combine map/filter with visual cards. Explain keys using changing lists.
Session 6	Mini project	90–120 min	Students build Student Dashboard with limited instructor intervention.

Suggested homework

- Rebuild the Student Dashboard without looking at the solution.
- Add a CourseCard component.
- Replace hard-coded data with a separate data.js file.
- Add at least two new props to an existing component.
- Explain the component tree on paper.

Reference: Recommended Project Structure as Projects Grow

A sensible starting point for later modules

```
src/
├── assets/
├── components/
│   ├── Header.jsx
│   ├── Footer.jsx
│   └── Button.jsx
├── pages/
│   ├── Home.jsx
│   └── About.jsx
├── data/
│   └── courses.js
├── App.jsx
├── App.css
├── index.css
└── main.jsx
```

Why this structure?

- **components/** stores reusable UI pieces.
- **pages/** can later contain larger page-level components.
- **data/** can hold local dummy data used during development.
- **assets/** can contain imported images and other source assets.
- **App.jsx** can compose the application instead of containing every detail.

Important architecture note

Do not turn this into a rigid law. Project structure should follow project complexity. A tiny practice app can be simple; a large application needs clearer boundaries.

End of Module 1

Students completing this module should now understand the basic React mental model: UI is built from components, components can receive data through props, JSX describes the UI, arrays can become lists, and the project is organized into files that each have a clear job.

Next module: deeper component patterns, event handling and state fundamentals.